Projeto Data Analytics — RA1 e RA2

Relatório do RA1 seguido pela implementação executável e documentada do RA2. A fonte original do RA1 é estática; no RA2, a chegada de eventos é simulada conforme o Cenário A da atividade.

PUCPR

PONTIFÍCIA UNIVERSIDADE CATÓLICA DO PARANÁ

DATA ANALYTICS / PROJETO BASEADO EM PROBLEMAS

RA1

Ingestão e pré-processamento dos dados

Modelo de documentação da entrega

Projeto RA1 — Ingestão e pré-processamento de eventos semiestruturados com Apache Spark

Curso e turma PUCPR • Bacharelado em Sistemas de Informação • Data Analytics (PJD117A); turma e período letivo não informados.

Professor ou professora Não informado nos arquivos fornecidos.

Equipe Integrantes e identificações acadêmicas não informados nos arquivos fornecidos.

Repositório Projeto disponível apenas no diretório local; URL, branch e commit não informados.

Entrega Cidade não informada • 21/10/2026

Este relatório documenta a implementação disponível em ingestao_spark.py e as evidências observadas na amostra local do projeto.

Problema e perguntas analíticas

Contexto e problema

O material contém eventos semiestruturados identificados por evento, usuário, tipo, valor e horário. O problema técnico é assegurar que esses registros sejam lidos, validados e preparados de forma consistente antes de análises posteriores. Os tipos click, purchase e view sugerem interações, mas os arquivos não identificam a aplicação, os usuários reais nem um contexto de negócio; portanto, não se atribui significado além do que está documentado.

Objetivo e recorte

Ingerir o JSON local, deduplicar por event_id, excluir registros sem user_id, com value negativo ou timestamp não interpretável e produzir a camada Bronze. O recorte observável é um arquivo de amostra com seis linhas, datas de 16/09/2026 e unidade evento; a cobertura geográfica, a população de origem e a representatividade não são informadas.

Perguntas analíticas

As perguntas a seguir orientam análises futuras; este RA documenta a ingestão e a preparação, sem formular conclusões de negócio.

ID	Pergunta a investigar	Dados necessários
P1	Como varia a quantidade de eventos por tipo na amostra?	event_type; event_id; registros após deduplicação.
P2	Como os valores se distribuem entre os tipos de evento?	event_type e value; estatísticas descritivas e domínio de value a confirmar.
P3	Quantos registros são repetidos ou excluídos por falhas de qualidade?	event_id, user_id, value e timestamp; contagens antes/depois e motivos de rejeição.

Viabilidade inicial

O JSON fornece as cinco variáveis de origem necessárias para contar eventos, comparar valores por tipo e identificar registros repetidos ou inválidos. A amostra é pequena (seis linhas), não tem fonte externa, licença ou contexto de negócio documentados e não permite generalização. O script não registra contagens de rejeição por regra; a linha inválida viola três regras simultaneamente, então os motivos não podem ser contabilizados como ocorrências independentes.

Fontes de dados e ingestão

Ficha da fonte

Identificação F1 — Amostra local de eventos (raw_events.json)

Origem e acesso Arquivo local data/raw_events.json, fornecido com o projeto. Autoria, instituição, URL, licença e data de publicação não informadas; acesso verificado em 21/10/2026.

Formato e volume JSON Lines: 6 objetos, 5 atributos; inclui 1 ocorrência repetida de event_id.

Cobertura Quatro horários válidos distintos em 16/09/2026; um registro tem data inválida. Geografia e população não informadas.

Condições de uso Não há licença ou restrição de uso indicada nos materiais fornecidos; confirmar com a pessoa responsável pela origem antes de redistribuir.

Relação com o problema

A fonte apoia as perguntas sobre volume por event_type, distribuição de value e qualidade de event_id/user_id/timestamp. Não contém descrição de negócio, localização geográfica ou documentação que permita interpretar value como moeda ou inferir comportamento de uma população.

Procedimento de ingestão

Etapa	Procedimento ou parâmetro	Evidência no repositório
1	Arquivo local data/raw_events.json; sem etapa de download no script.	data/raw_events.json
2	Spark lê JSON com schema explícito: String, Integer, String, Double, String.	ingestao_spark.py — schema_eventos e spark.read.schema(...).json(...).
3	show() exibiu 6 linhas carregadas; a execução completa terminou com sucesso.	ingestao_spark.py — df_bruto.show(); execução local verificada em 21/10/2026.

Resultado da ingestão

A leitura carregou as seis linhas do arquivo local data/raw_events.json, com schema explícito de cinco campos. Não há download ou etapa manual no script; o arquivo precisa existir nesse caminho relativo ao diretório de execução. A execução verificada exibiu os seis registros.

Compreensão e qualidade

Uma linha representa um evento registrado; a saída contém apenas eventos que passaram pelas regras de preparação.

event_id é a chave usada pelo script para remover repetições. Há uma duplicata idêntica (EVT-001) na amostra; unicidade não é garantida na origem e não há regra para resolver IDs repetidos com conteúdos divergentes.

Dicionário inicial dos dados

Atributo	Significado	Tipo e formato	Unidade ou domínio
event_id	Identificador do evento e chave usada na deduplicação.	String; IDs como EVT-001; duplicado na origem.	Identificador textual; unicidade não garantida na origem.
user_id	Identificador inteiro do usuário associado ao evento.	Integer; 101–105 na amostra.	Inteiro; nulo em 1 linha bruta e não nulo na saída.
event_type	Categoria do evento.	String; click, purchase ou view observados.	Categorias textuais observadas; domínio não validado pelo script.
value	Valor numérico associado ao evento; sem unidade descrita.	Double; -50.0 a 1250.75 na origem.	Número decimal; a saída mantém apenas value >= 0.
timestamp	Horário original do evento.	String; formato ISO 8601 com Z esperado.	Texto; um valor inválido na origem.
timestamp_corrigido	Horário convertido para timestamp Spark.	Timestamp; derivado do formato yyyy-MM-dd'T'HH:mm:ss'Z'.	Timestamp; todos válidos na saída.
data_ingestao	Instante em que a execução adiciona metadado de ingestão.	Timestamp; current_timestamp().	Timestamp de execução; varia entre execuções.

Diagnóstico de qualidade

ID	Verificação e regra	Resultado observado	Evidência
Q1	Duplicidade: event_id repetido; comparar chave no JSON.	1 ocorrência excedente em 6 linhas (EVT-001 duplicado); 0 repetidos na saída.	raw_events.json; dropDuplicates por event_id em ingestao_spark.py; CSV final.
Q2	Campos e domínio: user_id presente; value >= 0; event_type não nulo.	1 user_id nulo e 1 value negativo, ambos em EVT-003; 3 categorias event_type observadas sem nulos.	raw_events.json; filtros da etapa 3; o domínio não é validado explicitamente.
Q3	Formato: timestamp deve corresponder ao padrão ISO com Z.	1/6 timestamp inválido (EVT-003); após preparação, 4/4 timestamps convertidos.	raw_events.json; to_timestamp/filter e coluna timestamp_corrigido no CSV.

Impacto dos problemas identificados

A duplicata de EVT-001 reduz a contagem bruta e precisa ser removida. EVT-003 tem user_id ausente, value = -50 e timestamp inválido; as três falhas afetam a mesma linha, que é excluída. O significado de valores negativos não está documentado, então a regra value >= 0 deve ser confirmada se houver estornos ou ajustes. A ausência de proveniência e de métricas de rejeição limita auditoria e interpretação.

Limpeza e preparação

Registro das decisões

Decisão e diagnóstico D1 — Q1: duplicidade pela chave event_id

Problema e abrangência

Coluna event_id; seis linhas brutas contêm uma ocorrência repetida do EVT-001, com os mesmos valores nos demais campos. Não foram observados IDs nulos nesta amostra.

Tratamento e justificativa

O script aplica dropDuplicates pela chave event_id para manter uma ocorrência por ID. Isso remove uma linha excedente e preserva o evento no exemplo, pois as cópias são idênticas. Se futuramente IDs repetidos tiverem conteúdos diferentes, o registro mantido pode não ser determinístico; seria necessário definir prioridade temporal ou outra regra de negócio.

Implementação ingestao_spark.py — executar_pipeline_ingestao(), etapa 3 (dropDuplicates).

Validação após o tratamento

A comparação do JSON com a saída mostra EVT-001 uma vez; a saída tem 4 IDs distintos em 4 linhas. Evidências: data/raw_events.json e output/bronze_layer/dados_processados.csv; execução do script concluída em 21/10/2026.

Resumo das intervenções

Decisão	Ação executada	Impacto nos dados	Evidência
D1 — Q1	Remover repetição idêntica de EVT-001 com dropDuplicates por event_id.	6 para 5 registros antes das demais regras; 1 linha excedente removida.	ingestao_spark.py, etapa 3; JSON e CSV final.
D2 — Q2/ Q3	Filtrar user_id nulo, value < 0 e timestamp não convertido; adicionar timestamp_corrigido e data_ingestao.	EVT-003 removido (1 registro, com três falhas simultâneas); saída com 4 linhas e 7 colunas.	ingestao_spark.py, etapa 3; execução e output/bronze_layer/dados_processados.csv.

Conjunto de dados resultante

Arquivo ou conjunto final output/bronze_layer/dados_processados.csv — CSV gerado a partir da camada Bronze; 4 registros e 7 colunas.

Unidade de observação final Um evento válido e deduplicado por linha, com horário original, horário convertido e horário de ingestão.

Dicionário atualizado Seção “Dicionário inicial dos dados” neste relatório; inclui os cinco atributos de origem e os dois acrescentados na preparação.

Comparação antes e depois

Medida	Após ingestão	Após preparação	Explicação da diferença
Registros	6 linhas JSON brutas.	4 eventos válidos preparados.	Uma repetição removida e EVT-003 excluído por três regras; 6 → 5 → 4.
Atributos	5 campos no schema de origem.	7 colunas no CSV Bronze.	Adicionadas timestamp_corrigido e data_ingestao.
Duplicidade por event_id	1 ocorrência excedente entre 6 linhas (16,7% das linhas brutas).	0 ocorrências repetidas entre 4 linhas.	Deduplicação pela chave event_id; cópias de EVT-001 eram idênticas.

Verificações finais

Na saída verificada há 4 linhas e 7 colunas; event_id não se repete, user_id está preenchido, value é não negativo e timestamp_corrigido está preenchido em todas as linhas. O schema Spark define os cinco tipos de origem; as duas colunas de data são timestamps. A execução também exibiu os dados preparados e gravou o CSV.

event_type e nulidade de event_id não são validados explicitamente pelo código; estes testes descrevem a amostra observada, não uma garantia para entradas futuras.

Limitações e pendências

A amostra contém apenas seis linhas e não tem proveniência, licença, definição de negócio para value, população ou geografia documentadas. O código não grava rejeições nem contagens por regra, não valida event_id/event_type nulos, e dropDuplicates pode escolher arbitrariamente entre IDs conflitantes. toPandas() coleta os dados no driver, limitando escalabilidade; data_ingestao varia a cada execução. A execução apresentou avisos de compatibilidade do pandas 3.0.5 e usou fallback sem Arrow (PyArrow não instalado), mas terminou com sucesso.

LIMITE DESTA ENTREGAAgregações analíticas, indicadores, visualizações e conclusões pertencem aos próximos RAs. Aqui, demonstre a qualidade e a consistência dos dados preparados.

Organização e reprodutibilidade

Repositório e versão Não há repositório Git, URL, branch ou commit no material disponível; execução a partir do diretório local do projeto.

Ambiente Windows; Python 3.14.6; PySpark 4.2.0; pandas 3.0.5; Java instalado (versão não registrada). Ambiente virtual local .venv; não foi fornecido requirements.txt/pyproject.toml.

Localização dos artefatos

Artefato	Caminho ou forma de obtenção
Dados brutos	data/raw_events.json — arquivo local; sem obtenção externa.
Código de ingestão e preparação	ingestao_spark.py — executar a partir da raiz.
Dados preparados	output/bronze_layer/dados_processados.csv — gerado pelo script.
Documentação e dependências	Este relatório em output; .venv local; sem manifesto de dependências.

Passos para reprodução

Ordem	Instrução executável	Resultado esperado
1	Ativar .venv\Scripts\Activate.ps1; confirmar Python 3.14.6, PySpark 4.2.0 e pandas 3.0.5.	Ambiente virtual ativo; dependências disponíveis.
2	Usar o data/raw_events.json fornecido, mantendo o caminho relativo.	Arquivo JSON acessível na raiz do projeto.
3	Na raiz, executar: python ingestao_spark.py	Spark exibe ingestão, preparação e métricas sumárias no console.
4	Conferir output/bronze_layer/dados_processados.csv e contar linhas/colunas.	CSV com 4 registros e 7 colunas; nenhum event_id repetido na amostra.

Verificação da reprodução

Verificação realizada em 21/10/2026, no ambiente local descrito acima. Responsável nominal não informado. Executado `python ingestao_spark.py` a partir da raiz do projeto; processo terminou com sucesso e gerou o CSV com 4 linhas de dados. Houve avisos de Arrow/pandas, com fallback funcional.

Referências e anexos

Referências

Autoria não informada. Conjunto de amostra disponível em data/raw_events.json, formato JSON Lines, sem URL, versão ou licença declaradas. Acesso ao arquivo local: 21/10/2026.

Projeto RA1. Script de ingestão e preparação: ingestao_spark.py. Código local fornecido; execução verificada em 21/10/2026.

Conferência da entrega

☒ Problema, recorte e perguntas analíticas estão definidos.

☒ Fontes e procedimentos de ingestão estão documentados.

☒ A qualidade foi avaliada com evidências verificáveis.

☒ As decisões de limpeza estão justificadas e validadas.

☒ O conjunto de dados preparado está identificado.

☒ O repositório contém instruções suficientes para reprodução.

☒ Os campos-modelo foram preenchidos e os exemplos revisados.

Implementação reproduzível do RA1

As próximas células reproduzem a ingestão do JSON local, avaliação dos problemas de qualidade, limpeza e exportação da camada Bronze. A sessão Spark é mantida aberta para a execução do RA2 e encerrada apenas no final do notebook.

In [1]:
```
import os
import tempfile
import time
import urllib.request
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, DoubleType
from pyspark.sql.functions import col, count, mean, stddev, min, max, to_timestamp,
current_timestamp, window, expr

hadoop_dir = os.path.join(tempfile.gettempdir(), "hadoop")
bin_dir = os.path.join(hadoop_dir, "bin")
os.makedirs(bin_dir, exist_ok=True)
os.environ["HADOOP_HOME"] = hadoop_dir
os.environ["PATH"] = bin_dir + os.pathsep + os.environ.get("PATH", "")
winutils_path = os.path.join(bin_dir, "winutils.exe")
hadoop_dll_path = os.path.join(bin_dir, "hadoop.dll")
try:
    if not os.path.exists(winutils_path):
        urllib.request.urlretrieve("https://github.com/steveloughran/winutils/raw/master/hadoop-
3.0.0/bin/winutils.exe", winutils_path)
    if not os.path.exists(hadoop_dll_path):
        urllib.request.urlretrieve("https://github.com/steveloughran/winutils/raw/master/hadoop-
3.0.0/bin/hadoop.dll", hadoop_dll_path)
except Exception as error:
    print(f"Aviso ao preparar os binarios Hadoop: {error}")

spark = (
    SparkSession.builder
    .appName("Projeto_RA1_RA2")
    .config("spark.driver.bindAddress", "127.0.0.1")
    .config("spark.sql.shuffle.partitions", "4")
    .getOrCreate()
)
spark.sparkContext.setLogLevel("WARN")
print(f"Spark {spark.version} iniciado.")
```

```
C:\Users\adisj\RA2\.venv\Lib\site-packages\pyspark\testing\utils.py:127: FutureWarning: PySpark does
not yet fully support pandas >= 3.0.0. Some features may not work correctly. It is recommended to use
pandas < 3.0.0 for now.
  require_minimum_pandas_version()
Spark 4.2.0 iniciado.
```

Ingestão da base RA1

O arquivo `data/raw_events.json` é JSON Lines com seis registros. O schema explícito define os cinco campos de origem antes da leitura.

In [2]:
```
schema_eventos = StructType([
    StructField("event_id", StringType(), True),
    StructField("user_id", IntegerType(), True),
    StructField("event_type", StringType(), True),
    StructField("value", DoubleType(), True),
    StructField("timestamp", StringType(), True),
])

df_bruto = spark.read.schema(schema_eventos).json("data/raw_events.json")
print("Dados brutos ingeridos:")
df_bruto.show(truncate=False)
print("Total bruto:", df_bruto.count())
```

```
Dados brutos ingeridos:
+--------+-------+----------+-------+--------------------+
|event_id|user_id|event_type|value  |timestamp           |
+--------+-------+----------+-------+--------------------+
|EVT-001 |101    |click     |150.5  |2026-09-16T10:00:00Z|
|EVT-002 |102    |purchase  |890.0  |2026-09-16T10:05:00Z|
|EVT-003 |NULL   |click     |-50.0  |invalid_date        |
|EVT-001 |101    |click     |150.5  |2026-09-16T10:00:00Z|
|EVT-004 |104    |purchase  |1250.75|2026-09-16T10:15:00Z|
|EVT-005 |105    |view      |0.0    |2026-09-16T10:20:00Z|
+--------+-------+----------+-------+--------------------+

Total bruto: 6
```

Avaliação e preparação RA1

A amostra contém uma repetição de `EVT-001`. `EVT-003` tem usuário ausente, valor negativo e data inválida; por isso essa linha é excluída uma única vez após a deduplicação. As etapas resultam em 6 linhas brutas, 5 após deduplicar e 4 válidas na camada Bronze.

In [3]:
```
df_deduplicado = df_bruto.dropDuplicates(["event_id"])
df_com_timestamp = df_deduplicado.withColumn(
    "timestamp_corrigido",
    to_timestamp(col("timestamp"), "yyyy-MM-dd'T'HH:mm:ss'Z'"),
)
df_processado = (
    df_com_timestamp
    .filter(col("user_id").isNotNull())
    .filter(col("value") >= 0)
    .filter(col("timestamp_corrigido").isNotNull())
    .withColumn("data_ingestao", current_timestamp())
)

print(f"Linhas brutas: {df_bruto.count()}")
print(f"Após deduplicação: {df_deduplicado.count()}")
print(f"Após validação: {df_processado.count()}")
print("Dados preparados (camada Bronze):")
df_processado.show(truncate=False)
print("Resumo descritivo por tipo (evidência da implementação RA1):")
df_processado.groupBy("event_type").agg(
    count("event_id").alias("total_eventos"),
    mean("value").alias("media_valor"),
    stddev("value").alias("desvio_padrao_valor"),
    min("value").alias("valor_minimo"),
    max("value").alias("valor_maximo"),
).show(truncate=False)
```

```
Linhas brutas: 6
Após deduplicação: 5
```

```
Após validação: 4
Dados preparados (camada Bronze):
+--------+-------+----------+-------+--------------------+-------------------+-----------------------
---+
|event_id|user_id|event_type|value  |timestamp           |timestamp_corrigido|data_ingestao
|
+--------+-------+----------+-------+--------------------+-------------------+-----------------------
---+
|EVT-001 |101    |click     |150.5  |2026-09-16T10:00:00Z|2026-09-16 10:00:00|2026-10-21 20:19:09.999
757|
|EVT-002 |102    |purchase  |890.0  |2026-09-16T10:05:00Z|2026-09-16 10:05:00|2026-10-21 20:19:09.999
757|
|EVT-004 |104    |purchase  |1250.75|2026-09-16T10:15:00Z|2026-09-16 10:15:00|2026-10-21 20:19:09.999
757|
|EVT-005 |105    |view      |0.0    |2026-09-16T10:20:00Z|2026-09-16 10:20:00|2026-10-21 20:19:09.999
757|
+--------+-------+----------+-------+--------------------+-------------------+-----------------------
---+

Resumo descritivo por tipo (evidência da implementação RA1):
+----------+-------------+-----------+-------------------+------------+------------+
|event_type|total_eventos|media_valor|desvio_padrao_valor|valor_minimo|valor_maximo|
+----------+-------------+-----------+-------------------+------------+------------+
|purchase  |2            |1070.375   |255.08877131304703 |890.0       |1250.75     |
|view      |1            |0.0        |NULL               |0.0         |0.0         |
|click     |1            |150.5      |NULL               |150.5       |150.5       |
+----------+-------------+-----------+-------------------+------------+------------+
```

Exportação do conjunto preparado

O CSV Bronze contém os quatro eventos válidos e sete colunas, incluindo os horários corrigido e de ingestão. Ele é o ponto de partida documentado para o projeto.

In [4]:
```
os.makedirs("output/bronze_layer", exist_ok=True)
caminho_saida = "output/bronze_layer/dados_processados.csv"
df_processado.toPandas().to_csv(caminho_saida, index=False)
print(f"Camada Bronze salva em {caminho_saida}")
```

```
C:\Users\adisj\RA2\.venv\Lib\site-packages\pyspark\sql\pandas\conversion.py:298: FutureWarning: PySpa
rk does not yet fully support pandas >= 3.0.0. Some features may not work correctly. It is recommende
d to use pandas < 3.0.0 for now.
  require_minimum_pandas_version()
C:\Users\adisj\RA2\.venv\Lib\site-packages\pyspark\sql\pandas\conversion.py:348: UserWarning: toPanda
s attempted Arrow optimization because 'spark.sql.execution.arrow.pyspark.enabled' is set to true; ho
wever, failed by the reason below:
  [PACKAGE_NOT_INSTALLED] PyArrow >= 18.0.0 must be installed; however, it was not found.
Attempting non-optimization as 'spark.sql.execution.arrow.pyspark.fallback.enabled' is set to true.
  warn(msg)
Camada Bronze salva em output/bronze_layer/dados_processados.csv
C:\Users\adisj\RA2\.venv\Lib\site-packages\pyspark\sql\pandas\types.py:778: FutureWarning: PySpark do
es not yet fully support pandas >= 3.0.0. Some features may not work correctly. It is recommended to
use pandas < 3.0.0 for now.
  require_minimum_pandas_version()
C:\Users\adisj\RA2\.venv\Lib\site-packages\pyspark\sql\pandas\types.py:778: FutureWarning: PySpark do
es not yet fully support pandas >= 3.0.0. Some features may not work correctly. It is recommended to
use pandas < 3.0.0 for now.
  require_minimum_pandas_version()
```

RA2 — Processamento em streaming

Objetivo da análise

Monitorar quantos eventos de cada categoria do RA1 (`click` , `purchase` e `view`) chegam em cada intervalo de tempo. A análise demonstra recepção e atualização incremental, sem produzir conclusões de negócio.

Fonte e processamento

A base RA1 é estática. Para cumprir o Cenário A, o Spark usa a fonte `rate` para simular registros novos a cinco eventos por segundo. A regra artificial distribui os eventos entre as três categorias observadas no RA1; horários e IDs também são gerados para a demonstração.

In [5]:
```
df_stream_raw = (
    spark.readStream
    .format("rate")
    .option("rowsPerSecond", 5)
    .option("numPartitions", 1)
    .load()
)

df_eventos_stream = df_stream_raw.select(
    col("timestamp").alias("event_time"),
    expr("""
        CASE
            WHEN value % 3 = 0 THEN 'click'
            WHEN value % 3 = 1 THEN 'purchase'
            ELSE 'view'
        END
    """).alias("event_type"),
    (expr("value % 100 + 100")).alias("user_id"),
)
print("Schema dos eventos simulados:")
df_eventos_stream.printSchema()
```

```
Schema dos eventos simulados:
root
 |-- event_time: timestamp (nullable = true)
 |-- event_type: string (nullable = false)
 |-- user_id: long (nullable = true)
```

Janela temporal e agregação

Foi utilizada uma janela **Tumbling** de 10 segundos: janelas fixas, consecutivas e sem sobreposição. Não há deslocamento configurado. A agregação conta os registros por janela e tipo de evento. O modo `complete` publica a tabela agregada atualizada a cada micro-lote.

In [6]:
```
df_agregado = (
    df_eventos_stream
    .groupBy(
        window(col("event_time"), "10 seconds"),
        col("event_type"),
    )
    .count()
)
```

Recepção contínua e resultados

A saída em memória permite consultar as contagens enquanto o stream está ativo. As duas observações, feitas em momentos diferentes, mostram o comportamento conforme chegam novos registros.

In [7]:
```
query_stream = (
    df_agregado.writeStream
    .outputMode("complete")
    .format("memory")
    .queryName("ra2_resultados")
    .trigger(processingTime="2 seconds")
    .start()
)
print("Stream ativo; registros simulados estão chegando.")
```

```
Stream ativo; registros simulados estão chegando.
```

Primeira observação

Aguarda o primeiro micro-lote e exibe as contagens atuais. O processamento local pode demorar mais do que o intervalo configurado do trigger.

In [8]:
```
time.sleep(8)
spark.table("ra2_resultados").orderBy("window", "event_type").show(truncate=False)
```

```
+----------------------------------------+----------+-----+
|window                                  |event_type|count|
+----------------------------------------+----------+-----+
|{2026-10-21 20:19:10, 2026-10-21 20:19:20}|click     |9    |
|{2026-10-21 20:19:10, 2026-10-21 20:19:20}|purchase  |8    |
|{2026-10-21 20:19:10, 2026-10-21 20:19:20}|view      |8    |
+----------------------------------------+----------+-----+
```

Segunda observação

Após mais tempo de chegada, consulta novamente a tabela. Compare contagens e janelas com a primeira observação; o lote pode ainda estar sendo atualizado no instante da consulta.

In [9]:
```
time.sleep(8)
spark.table("ra2_resultados").orderBy("window", "event_type").show(truncate=False)
```

```
+----------------------------------------+----------+-----+
|window                                  |event_type|count|
+----------------------------------------+----------+-----+
|{2026-10-21 20:19:10, 2026-10-21 20:19:20}|click     |12   |
|{2026-10-21 20:19:10, 2026-10-21 20:19:20}|purchase  |12   |
|{2026-10-21 20:19:10, 2026-10-21 20:19:20}|view      |11   |
|{2026-10-21 20:19:20, 2026-10-21 20:19:30}|click     |12   |
|{2026-10-21 20:19:20, 2026-10-21 20:19:30}|purchase  |11   |
|{2026-10-21 20:19:20, 2026-10-21 20:19:30}|view      |12   |
+----------------------------------------+----------+-----+
```

Resultados e interpretação

Cada linha representa a contagem de uma categoria em uma janela de 10 segundos. Como os eventos são simulados continuamente, lotes posteriores atualizam as contagens e podem apresentar novas janelas. Os valores são artificiais; a relação com o RA1 está nas categorias de eventos usadas na amostra estática.

In [10]:
```
query_stream.stop()
print("Stream RA2 encerrado.")
spark.table("ra2_resultados").orderBy("window", "event_type").show(truncate=False)
spark.stop()
print("Sessão Spark encerrada.")
```

```
Stream RA2 encerrado.
+----------------------------------------+----------+-----+
|window                                  |event_type|count|
+----------------------------------------+----------+-----+
|{2026-10-21 20:19:10, 2026-10-21 20:19:20}|click     |12   |
|{2026-10-21 20:19:10, 2026-10-21 20:19:20}|purchase  |12   |
|{2026-10-21 20:19:10, 2026-10-21 20:19:20}|view      |11   |
|{2026-10-21 20:19:20, 2026-10-21 20:19:30}|click     |12   |
|{2026-10-21 20:19:20, 2026-10-21 20:19:30}|purchase  |11   |
|{2026-10-21 20:19:20, 2026-10-21 20:19:30}|view      |12   |
+----------------------------------------+----------+-----+
```